



Markov Chains (🤪^🍕)

CPMSoc Mathematics & QuantSoc

Andy, Jacob, Jeff, Justin, Vir

Attendance

Please scan the QR code; and let us know about any dietary requirements.



(For the pizza!)



Contents

Markov Chains (😋^🍕)

Introduction to Markov Chains

Markov Chain Monte Carlo

Hidden Markov Models

Closing Remarks

Introduction to Markov Chains

Markov Process

Markov Process

Definition

We say a process $X(t)$ is Markov if

$$\mathbb{E}[f(X(t)) \mid \mathcal{F}_s] = \mathbb{E}[f(X(t)) \mid \sigma(X(s))]$$

Markov Process

Definition

We say a process $X(t)$ is Markov if

$$\mathbb{E}[f(X(t)) \mid \mathcal{F}_s] = \mathbb{E}[f(X(t)) \mid \sigma(X(s))]$$

But what does that mean in practice? What even is a Markov chain?

Markov Chains

Definition

Let X_n denote the n^{th} state of the process. We say X is a Markov chain if it is a Markov process, that is

$$\mathbb{P}(X_{n+1} = x_{n+1} \mid X_0 = x_0, X_1 = x_1, \dots, X_n = x_n) = \mathbb{P}(X_{n+1} = x_{n+1} \mid X_n = x_n).$$

Markov Chains

Definition

Let X_n denote the n^{th} state of the process. We say X is a Markov chain if it is a Markov process, that is

$$\mathbb{P}(X_{n+1} = x_{n+1} \mid X_0 = x_0, X_1 = x_1, \dots, X_n = x_n) = \mathbb{P}(X_{n+1} = x_{n+1} \mid X_n = x_n).$$

To put it in words, to know the next state of the process, all we need is its the current state.

Markov Chains

Definition

Let X_n denote the n^{th} state of the process. We say X is a Markov chain if it is a Markov process, that is

$$\mathbb{P}(X_{n+1} = x_{n+1} \mid X_0 = x_0, X_1 = x_1, \dots, X_n = x_n) = \mathbb{P}(X_{n+1} = x_{n+1} \mid X_n = x_n).$$

To put it in words, to know the next state of the process, all we need is its the current state.

Example

If we play a game using coin-flips where heads gives you \$1 and tails makes you lose \$1, and we start at a balance of \$0, does it matter how we got to our balance after the 67th coin-flip?

Markov Chains

Example

Can we model roulette using a Markov chain?

Markov Chains

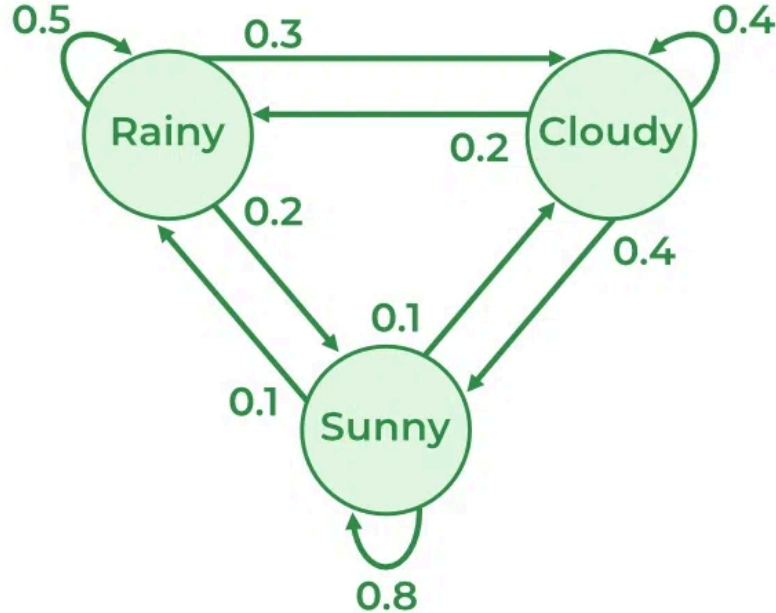
Example

Can we model roulette using a Markov chain?

No! Since the next outcome of the game does not depend on the outcome of the previous game.

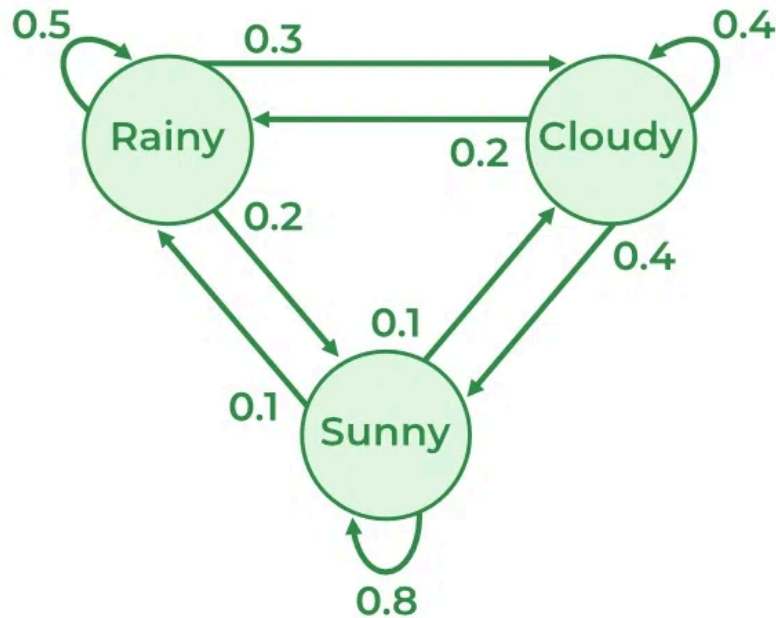
Modelling Markov Chains

A common way to model Markov chains is by using transition diagrams and matrices. Consider the following transition diagram. What does it represent?



Modelling Markov Chains

A common way to model Markov chains is by using transition diagrams and matrices. Consider the following transition diagram. What does it represent?



This is the transition matrix:

$$\begin{matrix} & \begin{matrix} R & C & S \end{matrix} \\ \begin{matrix} R \\ C \\ S \end{matrix} & \begin{pmatrix} 0.5 & 0.3 & 0.2 \\ 0.2 & 0.4 & 0.4 \\ 0.1 & 0.1 & 0.8 \end{pmatrix} \end{matrix}$$

Modelling Markov Chains

In general, we can label the probability of transitioning from state i to state j as P_{ij} .

Then, for any Markov chain, we have the transition matrix

$$P = \begin{pmatrix} P_{00} & P_{01} & P_{02} & \cdots \\ P_{10} & P_{11} & P_{12} & \cdots \\ P_{20} & P_{21} & P_{22} & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

with $\sum_j P_{ij} = 1$ for any i .

Chapman-Kolmogorov Equation

Suppose we wanted to calculate the probability of the process reaching a certain state, given a starting state, how can we do this?

Chapman-Kolmogorov Equation

Suppose we wanted to calculate the probability of the process reaching a certain state, given a starting state, how can we do this?

Chapman-Kolmogorov Equation

Let $P_{ij}^{(n)}$ denote the probability of the process being in state j after n steps, given it started in state i , and S be the state space. Then we have that

$$P_{ij}^{(n+m)} = \sum_{k \in S} P_{ik}^{(n)} P_{kj}^{(m)}.$$

Chapman-Kolmogorov Equation

Suppose we wanted to calculate the probability of the process reaching a certain state, given a starting state, how can we do this?

Chapman-Kolmogorov Equation

Let $P_{ij}^{(n)}$ denote the probability of the process being in state j after n steps, given it started in state i , and S be the state space. Then we have that

$$P_{ij}^{(n+m)} = \sum_{k \in S} P_{ik}^{(n)} P_{kj}^{(m)}.$$

Can you think about why this works?

Chapman-Kolmogorov Equation

Suppose we wanted to calculate the probability of the process reaching a certain state, given a starting state, how can we do this?

Chapman-Kolmogorov Equation

Let $P_{ij}^{(n)}$ denote the probability of the process being in state j after n steps, given it started in state i , and S be the state space. Then we have that

$$P_{ij}^{(n+m)} = \sum_{k \in S} P_{ik}^{(n)} P_{kj}^{(m)}.$$

Can you think about why this works? And, for transition matrices,

Matrix Form of the Chapman-Kolmogorov Equation

Let P be the transition matrix of the Markov chain. Then $P_{ij}^{(n)}$ is the $(i, j)^{th}$ entry of P^n .

State Spaces

Definition

We say states i and j communicate if $P_{ij}^{(n)} > 0$ and $P_{ji}^{(n)} > 0$ for some n , or equivalently, there exists a path on the Markov chain diagram between states i and j in both directions.

We say a state i is absorbing if $P_{ii} = 1$; equivalently, no states communicate with state i . If all states communicate with each other, then the Markov chain is called irreducible.

State Spaces

Definition

We say states i and j communicate if $P_{ij}^{(n)} > 0$ and $P_{ji}^{(n)} > 0$ for some n , or equivalently, there exists a path on the Markov chain diagram between states i and j in both directions.

We say a state i is absorbing if $P_{ii} = 1$; equivalently, no states communicate with state i . If all states communicate with each other, then the Markov chain is called irreducible.

Definition

We say a state is transient if it can be accessed only a finite amount of times, otherwise we say it is recurrent.

State Spaces

Definition

We say states i and j communicate if $P_{ij}^{(n)} > 0$ and $P_{ji}^{(n)} > 0$ for some n , or equivalently, there exists a path on the Markov chain diagram between states i and j in both directions.

We say a state i is absorbing if $P_{ii} = 1$; equivalently, no states communicate with state i . If all states communicate with each other, then the Markov chain is called irreducible.

Definition

We say a state is transient if it can be accessed only a finite amount of times, otherwise we say it is recurrent.

Definition

We say state i has period d if $P_{ii}^{(n)} = 0$ whenever d does not divide n . If a Markov chain has period 1 and all states are recurrent, we say it is ergodic.

State Spaces Example

$$\begin{bmatrix} 0 & 0.8 & 0.2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.5 & 0.5 & 0 & 0 & 0 \\ 0.6 & 0 & 0 & 0 & 0 & 0.4 & 0 \\ 0 & 0 & 0.6 & 0 & 0.4 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Stationary and Limiting Distributions

What if we want to know about the ‘average’ behaviour of a given Markov chain? And what might happen to it after an infinite number of steps? Does it always end up in some state?

Stationary and Limiting Distributions

What if we want to know about the ‘average’ behaviour of a given Markov chain? And what might happen to it after an infinite number of steps? Does it always end up in some state?

For simplicity, let’s assume that the state space is finite.

Definition

In a finite state Markov chain, let $\pi = (\pi_1 \ \pi_2 \ \cdots \ \pi_n)$ be a row vector with π_j representing the probability of being in state j at a given moment. If we have

$$\pi P = \pi, \quad \sum_{k \in S} \pi_k = 1,$$

then we say that π is the stationary distribution of our Markov chain.

Stationary and Limiting Distributions

What if we want to know about the ‘average’ behaviour of a given Markov chain? And what might happen to it after an infinite number of steps? Does it always end up in some state?

For simplicity, let’s assume that the state space is finite.

Definition

In a finite state Markov chain, let $\pi = (\pi_1 \ \pi_2 \ \cdots \ \pi_n)$ be a row vector with π_j representing the probability of being in state j at a given moment. If we have

$$\pi P = \pi, \quad \sum_{k \in S} \pi_k = 1,$$

then we say that π is the stationary distribution of our Markov chain.

But what does this represent?

Stationary and Limiting Distributions

Definition

We call

$$\pi_j = \lim_{k \rightarrow \infty} P_{ij}^{(k)}$$

the limiting probability that the process will be in state j . The row vector $\pi = (\pi_1 \ \pi_2 \ \cdots \ \pi_n)$ is then called the limiting distribution of the Markov chain.

Stationary and Limiting Distributions

Definition

We call

$$\pi_j = \lim_{k \rightarrow \infty} P_{ij}^{(k)}$$

the limiting probability that the process will be in state j . The row vector $\pi = (\pi_1 \ \pi_2 \ \cdots \ \pi_n)$ is then called the limiting distribution of the Markov chain.

Theorem

If the Markov chain is irreducible and ergodic, then the limiting distribution exists, is independent of the starting state i and satisfies

$$\pi P = \pi, \quad \sum_{k \in S} \pi_k = 1.$$

Markov Chain Monte Carlo

Introduction to Monte Carlo Methods

Monte Carlo Methods

Introduction to Monte Carlo Methods

Monte Carlo Methods

Definition

Class of experiments that undertake a frequentist approach in sampling events and then simply counting to approximate the probabilities of an event,

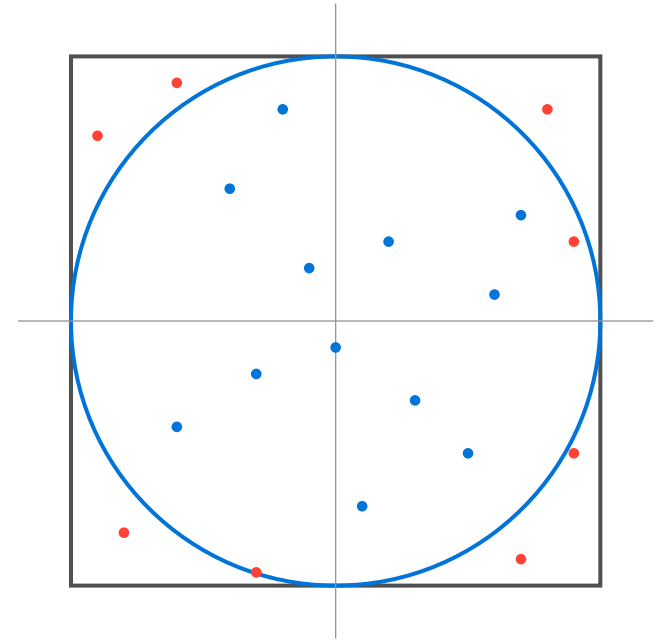
$$\Pr(\text{Event}) = \frac{\# \text{Favourable Outcomes}}{\# \text{Total Outcomes}}.$$

This probability can then also be used to approximate expected values.

Monte Carlo to Approximate Pi

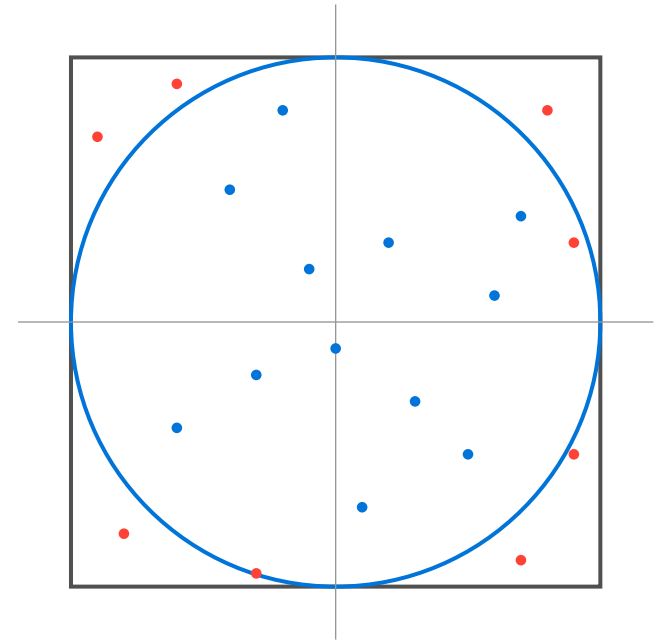
Monte Carlo to Approximate Pi

1. Randomly generates points to fit on a unit square with a unit circle embedded
2. Repeat this process many times
3. As a result $\frac{A_{\text{circle}}}{A_{\text{square}}} \approx \frac{\# \text{Points in circle}}{\# \text{Points}}$
4. This means we now have an approximation for π since $\frac{A_{\text{circle}}}{A_{\text{square}}} = \frac{\pi(1)^2}{(1)^2} = \pi$.



Monte Carlo to Approximate Pi

1. Randomly generates points to fit on a unit square with a unit circle embedded
2. Repeat this process many times
3. As a result $\frac{A_{\text{circle}}}{A_{\text{square}}} \approx \frac{\# \text{Points in circle}}{\# \text{Points}}$
4. This means we now have an approximation for π since $\frac{A_{\text{circle}}}{A_{\text{square}}} = \frac{\pi(1)^2}{(1)^2} = \pi$.



A more effective procedure would be NOT considering a quarter circle, as initially drafted since the points infinitesimally small.

Markov Chain Monte Carlo

IN MCMC, we generate a sequence of samples by constructing a Markov chain whose stationary distribution exactly matches our target distribution $f(x)$.

Markov Chain Monte Carlo

IN MCMC, we generate a sequence of samples by constructing a Markov chain whose stationary distribution exactly matches our target distribution $f(x)$.

But how do we decide which state to jump to next?

Definition

A **proposal function** (or proposal distribution), denoted $g(x^C \mid x_t)$, is a conditional probability distribution used to suggest a candidate state x^C given the current state x_t .

Markov Chain Monte Carlo

It should be relatively easy to sample - note that our choice of g greatly dictates how efficiently our Markov chain “explores” the state space without getting stuck.

Markov Chain Monte Carlo

It should be relatively easy to sample - note that our choice of g greatly dictates how efficiently our Markov chain “explores” the state space without getting stuck.

Metropolis-Hastings Acceptance

To guarantee the chain converges to our desired target $f(x)$, candidates are accepted or rejected based on a ratio α . While the original Metropolis method assumes a symmetric proposal ($g(x^C | x_t) = g(x_t | x^C)$), Hastings generalized this for asymmetric proposals:

$$\alpha = \min \left(1, \frac{f(x^C)g(x_t | x^C)}{f(x_t)g(x^C | x_t)} \right)$$

Demonstration of Markov Chain Monte Carlo

For a Toy Demonstration let us consider the following application

Demonstration of Markov Chain Monte Carlo

For a Toy Demonstration let us consider the following application

Metropolis Algorithm

Let $f(x) = \lambda p(x)$ where p is the pdf you are expecting, $\lambda \in \mathbb{R}$

1. **Initialisation:** Select first observation x_t , proposal function g
2. **Experimentation** For $t = 0$ to n trials,
 - Compute candidate $x_t^C = g(x|x_t)$ through a trial
 - Compute $\alpha = \min\left(1, \frac{f(x_t^C)}{f(x_t)}\right)$
 - Using decision variable $U = \text{Uniform}(0, 1)$, $x_{t+1} = \mathbb{1}_{U \leq \alpha} x_t^C + \mathbb{1}_{U > \alpha} x_t$
3. Now you have a sample $(x_0, \dots, x_n)$ through which you can estimate $\mathbb{E}, \mathbb{V}$

Demonstration of Markov Chain Monte Carlo

Example

Bond is gambling in a Casino in Monte Carlo, playing a newly launched game of Justin-Poker. Since the game is newly launched, Bond assumes his wins $X \sim \text{Uniform}(-1, 1)$ per game. Using Metropolis' and proposal function $g(x_t^C | x) = x + Y$ where $Y = \text{Uniform}(-\frac{1}{2}, \frac{1}{2})$, estimate $\mathbb{E}[X]$.

Demonstration of Markov Chain Monte Carlo

Example

Bond is gambling in a Casino in Monte Carlo, playing a newly launched game of Justin-Poker. Since the game is newly launched, Bond assumes his wins $X \sim \text{Uniform}(-1, 1)$ per game. Using Metropolis' and proposal function $g(x_t^C | x) = x + Y$ where $Y = \text{Uniform}(-\frac{1}{2}, \frac{1}{2})$, estimate $\mathbb{E}[X]$.

1. **Initialisation:** Let $f(x) = 2p(x)$ as computing α cancels $\frac{1}{2}$
2. **Experimentation:** Say validly, $x_0 = a$, $x_0^C = a + b$, $\alpha = 1 + \frac{b}{a}$.
3. Compute decision var U , if $U \leq \alpha$, $x_1 = a + b$, otherwise $x_1 = a$
4. Repeat for say n trials then $\mathbb{E}[X] = \frac{1}{n} \sum_{i=0}^n x_i$.

Demonstration of Markov Chain Monte Carlo

Using some simple Python code, we ran a trial:

i	x_t	x_t^C	α	U	accept	x_{t+1}
0	0.0000	0.0488	1.0000	0.7152	True	0.0488
1	0.0488	0.1516	1.0000	0.5449	True	0.1516
2	0.1516	0.0752	1.0000	0.6459	True	0.0752
3	0.0752	0.0128	1.0000	0.8918	True	0.0128
4	0.0128	0.4765	1.0000	0.3834	True	0.4765
5	0.4765	0.7682	1.0000	0.5289	True	0.7682
6	0.7682	0.8363	1.0000	0.9256	True	0.8363
7	0.8363	0.4073	1.0000	0.0871	True	0.4073
8	0.4073	-0.0725	1.0000	0.8326	True	-0.0725
9	-0.0725	0.2057	1.0000	0.8700	True	0.2057
10	0.2057	0.6843	1.0000	0.7992	True	0.6843
11	0.6843	0.6458	1.0000	0.7805	True	0.6458
12	0.6458	0.2640	1.0000	0.6399	True	0.2640
13	0.2640	-0.0926	1.0000	0.9447	True	-0.0926
14	-0.0926	-0.0708	1.0000	0.4147	True	-0.0708
15	-0.0708	-0.3062	1.0000	0.7742	True	-0.3062
16	-0.3062	-0.3501	1.0000	0.5684	True	-0.3501
17	-0.3501	-0.8313	1.0000	0.6176	True	-0.8313
18	-0.8313	-0.7192	1.0000	0.6169	True	-0.7192
19	-0.7192	-0.2754	1.0000	0.6818	True	-0.2754

```
Samples: [ 0.      0.0488  0.1516  0.0752  0.0128  0.4765  0.7682  0.8363  (
-0.0725  0.2057  0.6843  0.6458  0.264 -0.0926 -0.0708 -0.3062 -0.3501
-0.8313 -0.7192 -0.2754]
Estimated E[X] = 1/n * sum(x_i): 0.08849547290129424
```

Complex Demonstration of Markov Chain Monte Carlo

Setting

Complex Demonstration of Markov Chain Monte Carlo

Setting

Consider a graph $G = (V, E)$ describing a series of $|V|$ nodes that each have a state $s(V)$ where $s(V) \in \{1, 2, \dots, q\}$.

Complex Demonstration of Markov Chain Monte Carlo

Setting

Consider a graph $G = (V, E)$ describing a series of $|V|$ nodes that each have a state $s(V)$ where $s(V) \in \{1, 2, \dots, q\}$.

We define the probability of two vertices being marked by an edge $e \in E$ as $p = 1 - \beta$ where $\beta \sim \text{Uniform}(0, 1)$ generally (like proposal function).

Complex Demonstration of Markov Chain Monte Carlo

Setting

Consider a graph $G = (V, E)$ describing a series of $|V|$ nodes that each have a state $s(V)$ where $s(V) \in \{1, 2, \dots, q\}$.

We define the probability of two vertices being marked by an edge $e \in E$ as $p = 1 - \beta$ where $\beta \sim \text{Uniform}(0, 1)$ generally (like proposal function).

Let us also note the probability of $v \in V$ not changing its state as $\frac{1}{q}$.

Complex Demonstration of Markov Chain Monte Carlo

Setting

Consider a graph $G = (V, E)$ describing a series of $|V|$ nodes that each have a state $s(V)$ where $s(V) \in \{1, 2, \dots, q\}$.

We define the probability of two vertices being marked by an edge $e \in E$ as $p = 1 - \beta$ where $\beta \sim \text{Uniform}(0, 1)$ generally (like proposal function).

Let us also note the probability of $v \in V$ not changing its state as $\frac{1}{q}$.

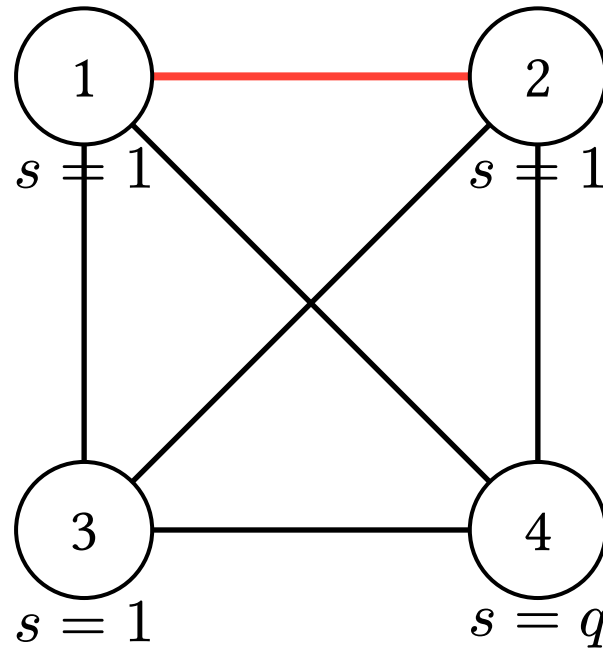
Now in this setting, each v undergoes a re-determination of its $s(v)$ at each discrete point in time t , and then an edge with a neighbour $n \in N(v)$ is marked with probability q if they share the same state.

Complex Demonstration of Markov Chain Monte Carlo

A Diagram

Complex Demonstration of Markov Chain Monte Carlo

A Diagram



At $t = 0$. $s(v)_t = \frac{1}{q}s(v)_{t-1} + \left(1 - \frac{1}{q}\right)\mathbb{E}[s(v) : s(v) \neq s(v)_{t-1}]$, red with p

Complex Demonstration of Markov Chain Monte Carlo

What is the Point of this?

Complex Demonstration of Markov Chain Monte Carlo

What is the Point of this?

1. Modelling the ‘spins’ of atoms in heated metals

Complex Demonstration of Markov Chain Monte Carlo

What is the Point of this?

1. Modelling the ‘spins’ of atoms in heated metals
2. Modelling colour pixel changes on a TV screen when there is no deterministic algorithm

Complex Demonstration of Markov Chain Monte Carlo

What is the Point of this?

1. Modelling the ‘spins’ of atoms in heated metals
2. Modelling colour pixel changes on a TV screen when there is no deterministic algorithm
3. Modelling the states of each ‘roll’ of a slot machine and their matching

Complex Demonstration of Markov Chain Monte Carlo

What is the Point of this?

1. Modelling the 'spins' of atoms in heated metals
2. Modelling colour pixel changes on a TV screen when there is no deterministic algorithm
3. Modelling the states of each 'roll' of a slot machine and their matching
4. General Crowd behaviour and clustering

Complex Demonstration of Markov Chain Monte Carlo

What is the Point of this?

1. Modelling the ‘spins’ of atoms in heated metals
2. Modelling colour pixel changes on a TV screen when there is no deterministic algorithm
3. Modelling the states of each ‘roll’ of a slot machine and their matching
4. General Crowd behaviour and clustering

This is a **markov process** as the state transition probabilities are independent of previous states, and similarly with the matching.

Complex Demonstration of Markov Chain Monte Carlo

Markov Chain Monte Carlo in this Setting

Complex Demonstration of Markov Chain Monte Carlo

Markov Chain Monte Carlo in this Setting

Swendsen-Wang Algorithm (modified)

At each time $t > 0$:

- 1. Unmark all 'marked' edges*
- 2. Undergo state change*
- 3. For each unmarked edge, apply marking, $\Pr(\text{mark}) = p$ if same s*
- 4. Now store information about the number of connected components with marked edges connecting them only and the size of each.*

Complex Demonstration of Markov Chain Monte Carlo

What is the Value of this?

Complex Demonstration of Markov Chain Monte Carlo

What is the Value of this?

With this information we know can compute the probability of vertices being in a certain state, and clustering in that state.

Complex Demonstration of Markov Chain Monte Carlo

What is the Value of this?

With this information we know can compute the probability of vertices being in a certain state, and clustering in that state.

A simple example on the next slide will lend clarity.

Complex Demonstration of Markov Chain Monte Carlo

A Simple Example

Complex Demonstration of Markov Chain Monte Carlo

A Simple Example

Example

Tom, Jeff, Justin are going to a casino in Monte Carlo. They find a slot machine and each choose a pattern. Out of the three patterns, Tom chooses 'Shield of Aaron Fox' (S), Jeff chooses 'Andy's Hat' (A) and Justin chooses 'Master Luke's Sword' (M). Who is the most likely to win if there are 3 rolls, each roll is equally likely to change, and the probability of two rolls aligning is $p = 1 - \beta$ where $\beta \sim \text{Uniform}(0, \frac{1}{2})$. At $t = 0$ run, ASM pattern.

Complex Demonstration of Markov Chain Monte Carlo

Breaking Down This Example

Complex Demonstration of Markov Chain Monte Carlo

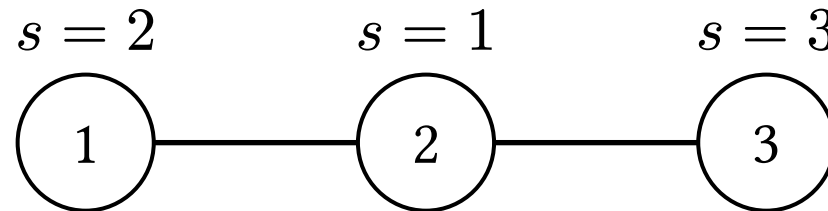
Breaking Down This Example

We can model this with the following diagram and information, where $s = 1$ denotes 'S', $s = 2$ denotes 'A', $s = 3$ denotes 'M', p is the marking probability.

Complex Demonstration of Markov Chain Monte Carlo

Breaking Down This Example

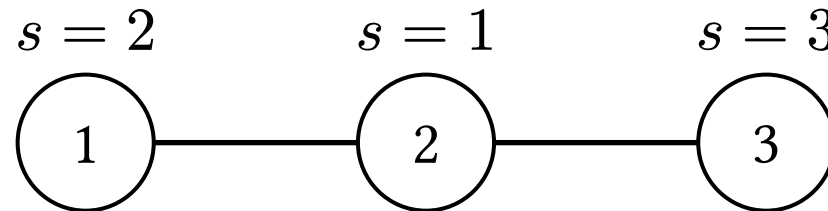
We can model this with the following diagram and information, where $s = 1$ denotes 'S', $s = 2$ denotes 'A', $s = 3$ denotes 'M', p is the marking probability.



Complex Demonstration of Markov Chain Monte Carlo

Breaking Down This Example

We can model this with the following diagram and information, where $s = 1$ denotes 'S', $s = 2$ denotes 'A', $s = 3$ denotes 'M', p is the marking probability.



The only reason we have 2 edges is since we only need two consecutive alignments on the slot machine.

Complex Demonstration of Markov Chain Monte Carlo

Conducting the Test for $n = 100$ trials

Complex Demonstration of Markov Chain Monte Carlo

Conducting the Test for $n = 100$ trials

Using some simple Python coding:

Trial	Node 1	Node 2	Node 3	Status
0	A	S	M	Initial
1	M	S	A	-
2	A	S	S	-
3	S	M	M	-
4	A	A	A	Matched A
5	S	S	S	Matched S
...	...	...	...	...
96	S	S	S	Matched S
97	M	A	A	-
98	A	A	A	Matched A
99	M	M	S	-
100	S	S	S	Matched S

Complex Demonstration of Markov Chain Monte Carlo

Analysing Results

Complex Demonstration of Markov Chain Monte Carlo

Analysing Results

=====

Results after 100 Swendsen-Wang sweeps:

=====

Probability of S-S-S (Tom) : 0.130

Probability of A-A-A (Jeff) : 0.140

Probability of M-M-M (Justin) : 0.110

From observation it seems Jeff's chance of winning are more likely but note that each state in just this scenario (a single 'S', 'A', 'M') is equally likely to occur (not in all cases), so only thing we can say is the chance of winning given all match in general is $0.11 \leq \Pr(\text{all match}) \leq 0.14$ roughly.

Hidden Markov Models

From Markov Chains to Hidden Markov Models

We have seen that Markov chains model systems whose future depends only on their current state.

From Markov Chains to Hidden Markov Models

We have seen that Markov chains model systems whose future depends only on their current state.

However, Markov chains assume that the state of the system is directly observable.

From Markov Chains to Hidden Markov Models

We have seen that Markov chains model systems whose future depends only on their current state.

However, Markov chains assume that the state of the system is directly observable.

In many real-world applications this assumption is unrealistic.

We often observe the consequences of a state, rather than the state itself.

From Markov Chains to Hidden Markov Models

Example

1. *Financial markets:*
 - *Hidden state: Bull market / Bear market*
 - *Observation: Asset returns*
2. *Weather:*
 - *Hidden state: Current Season*
 - *Observation: Weather Conditions*

In each case the underlying state exists, but cannot be directly observed, rather we must infer it.

Motivation

Example

Suppose we observe the sequence

Sunny, Rainy, Sunny, Sunny, Rainy, Rainy, Sunny, Rainy

Motivation

Example

Suppose we observe the sequence

Sunny, Rainy, Sunny, Sunny, Rainy, Rainy, Sunny, Rainy

We might not care about the observed weather, but are rather interested in a hidden climate regime, where we must infer from the observations above.

Motivation

Example

Suppose we observe the sequence

Sunny, Rainy, Sunny, Sunny, Rainy, Rainy, Sunny, Rainy

We might not care about the observed weather, but are rather interested in a hidden climate regime, where we must infer from the observations above.

Suppose we ‘know’ that swapping from a Dry Season to a Wet season is likely to occur, and that it’s unlikely for the opposite event to occur within the

season. What would your answer be here? What if its unlikely for seasons to swap? How does your answer differ?

Hidden Markov Models

Definition

A Hidden Markov Model (HMM) consists of

- 1. A sequence of hidden states (X_t) forming a Markov chain*
- 2. A sequence of observations (Y_t)*
- 3. Emission probabilities describing how observations are generated from hidden states*

Hidden Markov Models

Definition

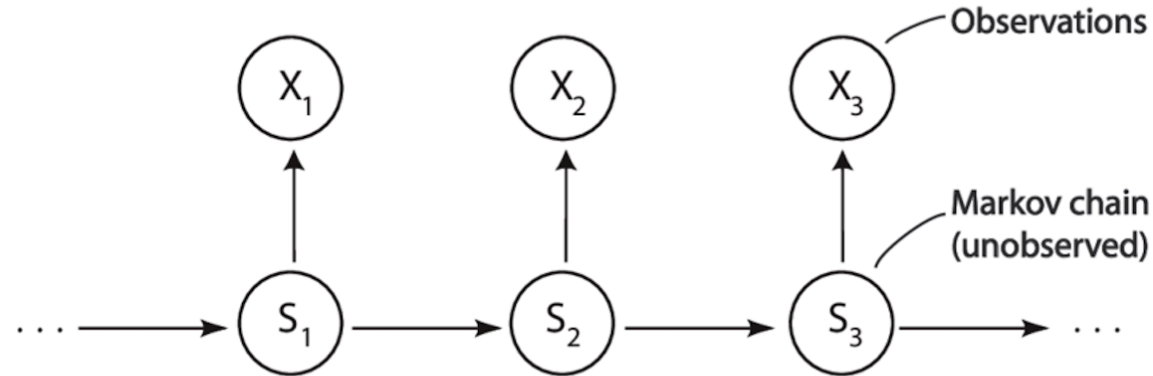
A Hidden Markov Model (HMM) consists of

- 1. A sequence of hidden states (X_t) forming a Markov chain*
- 2. A sequence of observations (Y_t)*
- 3. Emission probabilities describing how observations are generated from hidden states*

The hidden states evolve according to the Markov property, while observations provide indirect information about those states.

Hidden Markov Models

The key difference from a Markov chain is the introduction of observations.



We observe (X_t) but not (S_t) .

Hidden Markov Models

An HMM is specified by three probability distributions:

1. Initial probabilities

$$\pi_i = \mathbb{P}(X_1 = i)$$

1. Transition probabilities

$$A_{ij} = \mathbb{P}(X_t = j \mid X_{t-1} = i)$$

1. Emission probabilities

$$B_{jk} = \mathbb{P}(Y_t = k \mid X_t = j)$$

A Simple Example

Imagine somebody repeatedly flips one of two coins.

1. Coin A prefers heads.
2. Coin B prefers tails.

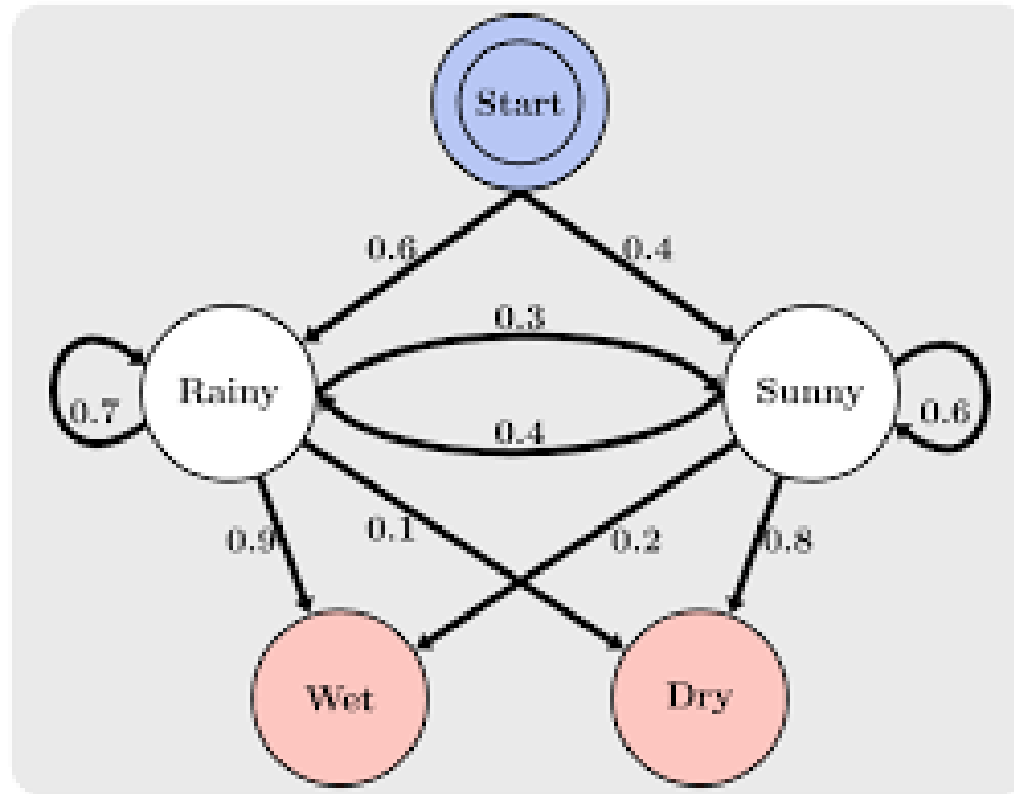
We observe only the outcomes:

H, T, T, T, H, T, T, T, H, T, H, H, H, H ...

We do not know which coin was used at each step.

The coin sequence forms the hidden states.

A Visual Example



Inference

Once we observe a sequence of data, several questions arise.

Inference

Once we observe a sequence of data, several questions arise.

1. What is the probability of the observations?
2. What is the current hidden state?
3. What is the most likely sequence of hidden states?

Inference

Once we observe a sequence of data, several questions arise.

1. What is the probability of the observations?
2. What is the current hidden state?
3. What is the most likely sequence of hidden states?

These questions lead to algorithms such as

1. Forward Algorithm
2. Viterbi Algorithm

The Model: Weather $\rightarrow$ Mood

We can't see the **weather**; we only see someone's **mood** each day.

- Hidden state x_t : weather — Sunny or Rainy
- Observation y_t : mood — Happy or Grumpy

$$\pi = (0.6, 0.4), \quad A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}, \quad B = \begin{pmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{pmatrix}$$

The Model: Weather $\rightarrow$ Mood

We can't see the **weather**; we only see someone's **mood** each day.

- Hidden state x_t : weather — Sunny or Rainy
- Observation y_t : mood — Happy or Grumpy

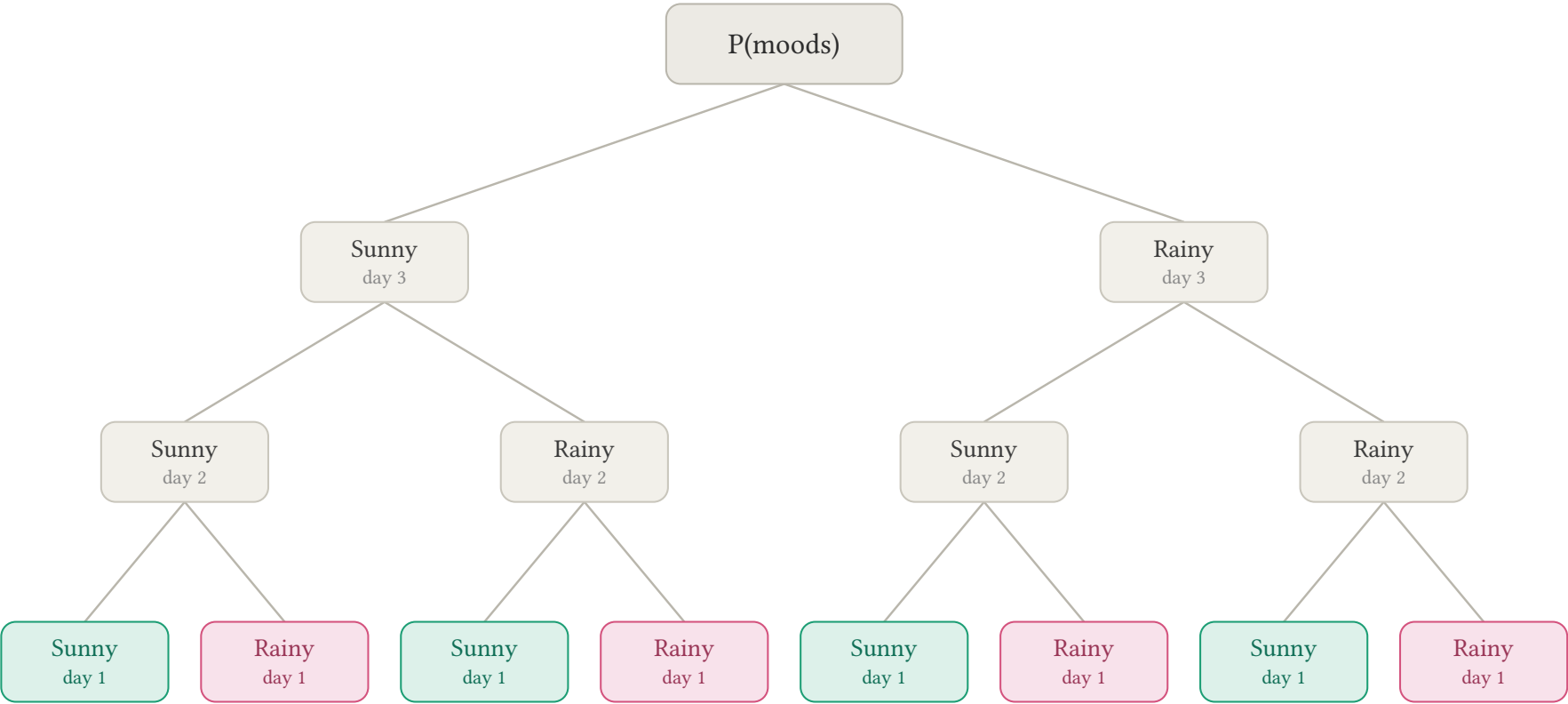
$$\pi = (0.6, 0.4), \quad A = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix}, \quad B = \begin{pmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{pmatrix}$$

(rows Sunny/Rainy; A cols Sunny/Rainy, B cols Happy/Grumpy.)

The moods we observe: **Happy, Grumpy, Grumpy**.

How Likely Were Those Moods?

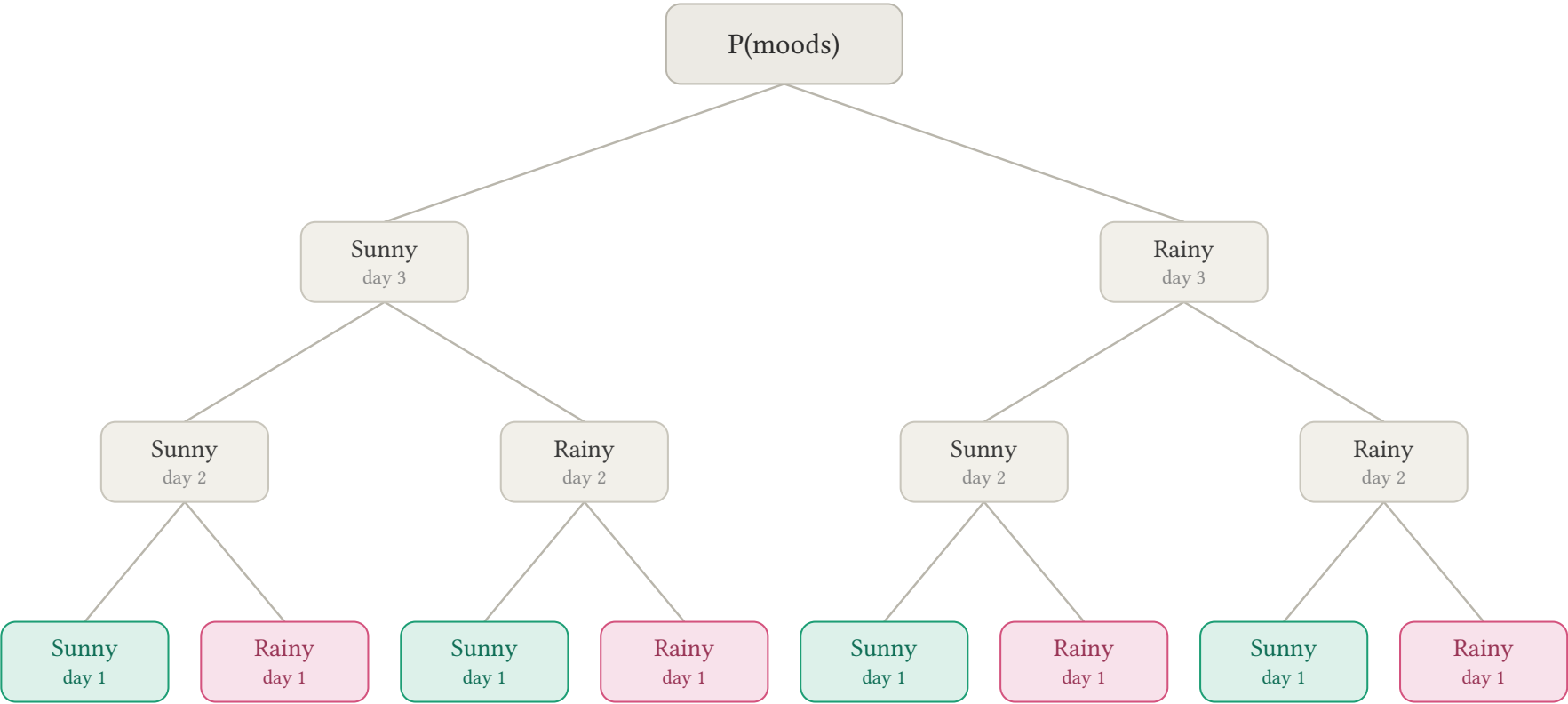
To get $\mathbb{P}(\text{moods})$ we must add up over **every** possible weather history. Written out, the choices branch into a tree:



Same colour = the exact same subproblem, recomputed. 8 paths here – 2^T in general.

How Likely Were Those Moods?

To get $\mathbb{P}(\text{moods})$ we must add up over **every** possible weather history. Written out, the choices branch into a tree:

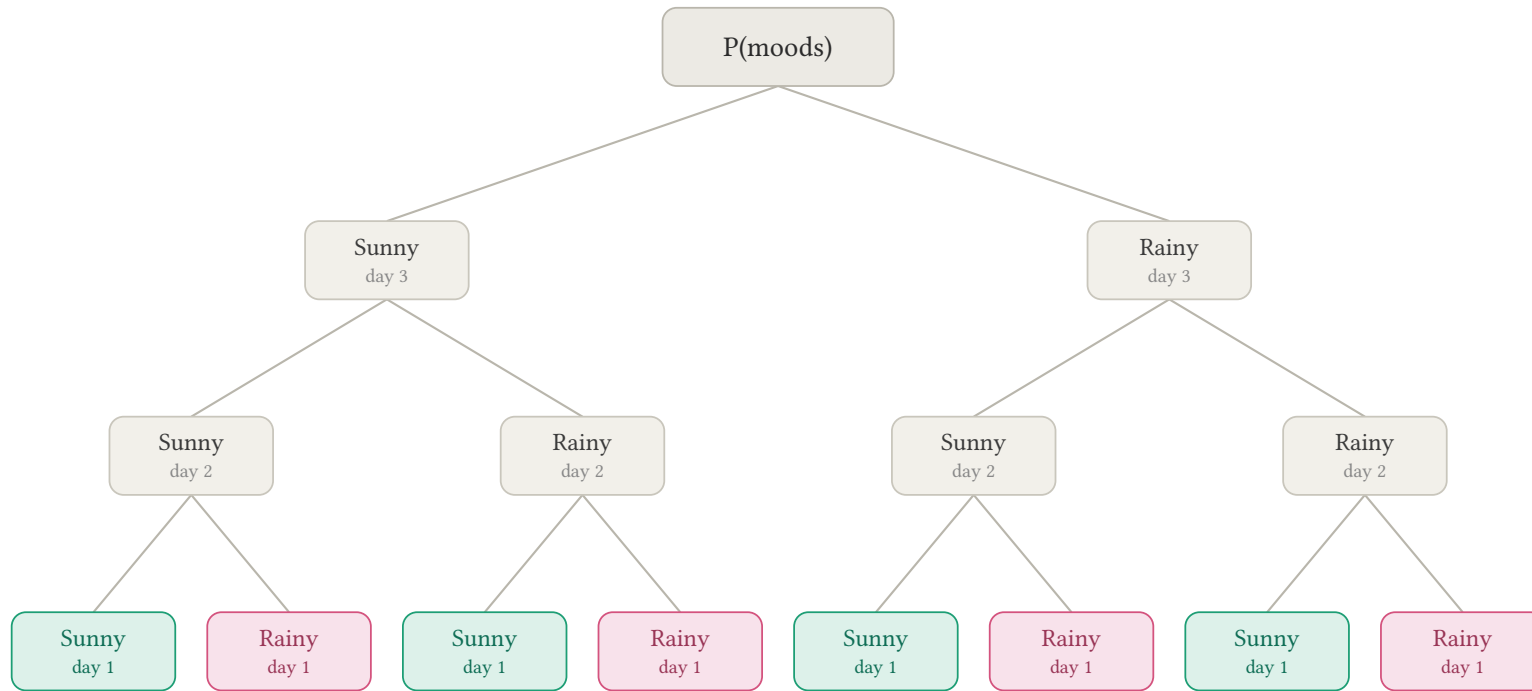


Same colour = the exact same subproblem, recomputed. 8 paths here – 2^T in general.

Each day doubles the branches — 2^T histories. Hopeless to enumerate directly.

The Tree Wastes Work

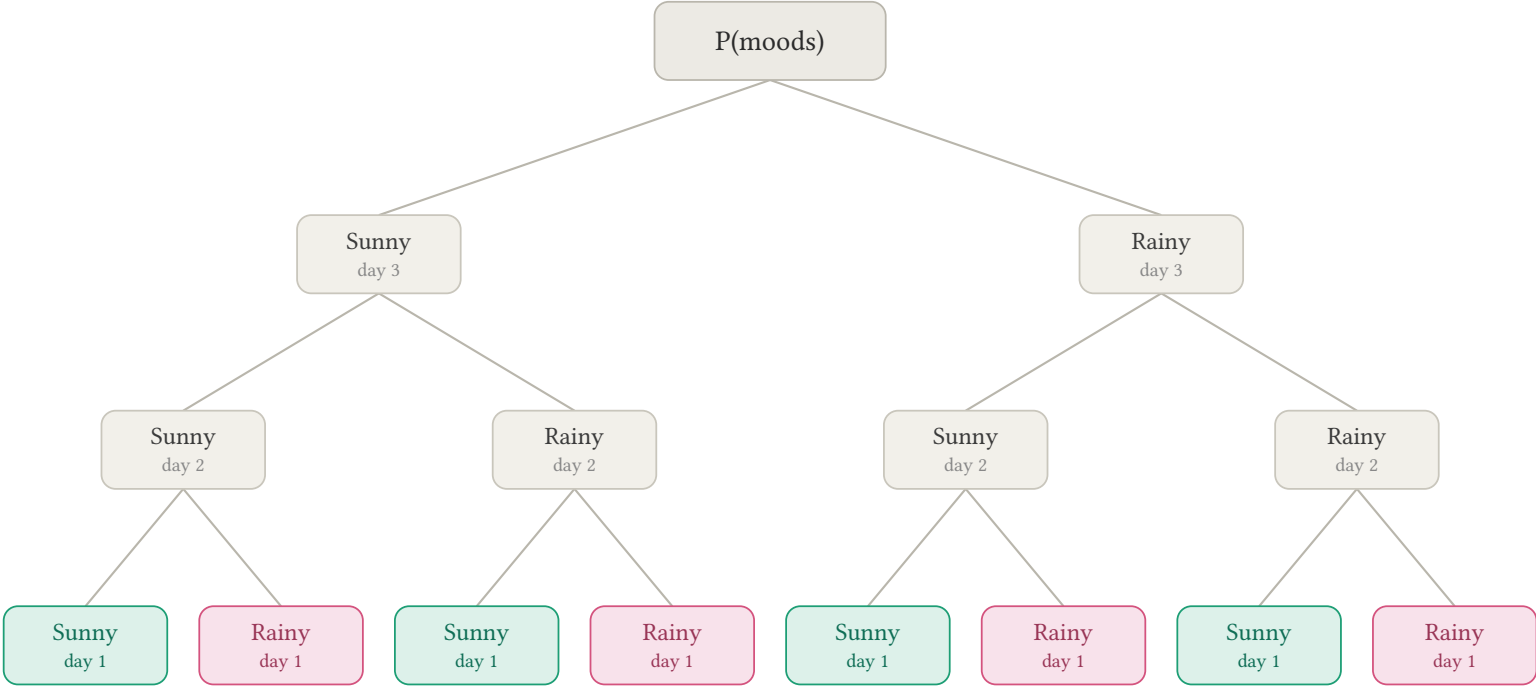
Look at the colours: the **same** subproblem — “Sunny on day 1” — is recomputed again and again across different branches.



Same colour = the exact same subproblem, recomputed. 8 paths here — 2^T in general.

The Tree Wastes Work

Look at the colours: the **same** subproblem — “Sunny on day 1” — is recomputed again and again across different branches.



Same colour = the exact same subproblem, recomputed. 8 paths here — 2^T in general.

Nothing about “Sunny on day 1” changes between branches. We should compute it **once** and reuse it.

Dynamic Programming: Fold the Tree

Store each subproblem once, and the exponential tree folds into a small grid — just **two** values per day, carried forward:

	day 1	day 2	day 3
Sunny	$\alpha_1(S)$	$\alpha_2(S)$	$\alpha_3(S)$
Rainy	$\alpha_1(R)$	$\alpha_2(R)$	$\alpha_3(R)$

$$\alpha_t(k) = \left[\sum_x \alpha_{t-1}(x) A_{xk} \right] B_{k,y_t}$$

Dynamic Programming: Fold the Tree

Store each subproblem once, and the exponential tree folds into a small grid — just **two** values per day, carried forward:

	day 1	day 2	day 3
Sunny	$\alpha_1(S)$	$\alpha_2(S)$	$\alpha_3(S)$
Rainy	$\alpha_1(R)$	$\alpha_2(R)$	$\alpha_3(R)$

$$\alpha_t(k) = \left[\sum_x \alpha_{t-1}(x) A_{xk} \right] B_{k,y_t}$$

Cost drops from 2^T to $O(N^2T)$. That reuse **is** the dynamic programming.

Viterbi: Keep the Best Branch

Same folded grid — but to find the **single most likely** weather story, at each cell keep the **best** incoming branch instead of the sum, and remember it.

$$V_t(k) = \max_x [V_{t-1}(x) A_{xk}] B_{k,y_t}$$

Viterbi: Keep the Best Branch

Same folded grid — but to find the **single most likely** weather story, at each cell keep the **best** incoming branch instead of the sum, and remember it.

$$V_t(k) = \max_x [V_{t-1}(x) A_{xk}] B_{k,y_t}$$

Sum $\rightarrow$ “how likely overall”; max $\rightarrow$ “the single best story”.

Viterbi by Hand: Start

Moods **Happy, Grumpy, Grumpy**. Day 1 is just start $\times$ mood:

$$V_1(S) = 0.6 \cdot 0.8 = 0.48, \quad V_1(R) = 0.4 \cdot 0.3 = 0.12$$

Viterbi by Hand: Start

Moods **Happy, Grumpy, Grumpy**. Day 1 is just start $\times$ mood:

$$V_1(S) = 0.6 \cdot 0.8 = 0.48, \quad V_1(R) = 0.4 \cdot 0.3 = 0.12$$

Day 2, the Rainy cell (today's mood is Grumpy):

$$V_2(R) = \max(0.48 \cdot 0.3, 0.12 \cdot 0.6) \cdot 0.7 = 0.144 \cdot 0.7 = 0.1008$$

The winner came from **Sunny**, so we store an arrow back to Sunny.

Closing Remarks

The End.

Thanks for coming! We hope you enjoyed 😊, pizza is coming.

Please complete the
feedback form 🙄

More mathematics
workshops to come!!!

Feel free to try the
worksheets and discuss!!!



Feedback Form

